

WebSocket 模块 详细报告

一、需求与设计

1.1 功能需求

需求编号	需求描述	优先级
WS-01	实时消息推送 (Chat WebSocket)	P0
WS-02	WebRTC 信令中继 (Voice WebSocket)	P0
WS-03	多标签页支持 (同一用户多Session广播)	P1
WS-04	HTTP Session 桥接 (认证信息传递)	P0
WS-05	Reaction/Recall 实时事件推送	P1
WS-06	群邀请 WebSocket 通知	P1
WS-07	通话信令通知 (invite/accepted/rejected/ended)	P0

1.2 设计决策

- 1. 两个独立 **WebSocket 端点** — `/ws/chat` (实时消息) 和 `/ws/voice` (信令中继)，职责分离
- 2. **HTTP Session 桥接** — 通过 `HttpSessionConfigurator` 将 Servlet `HttpSession` 传递给 WebSocket Session
- 3. **多标签页扇出** — `SocketRegistry` 用 `Set<Session>` 存储同一用户的多个连接
- 4. **优雅降级** — WebSocket 失败时前端自动降级到 REST (详见 Chat 模块)
- 5. **无服务器端消息队列** — WebSocket 是纯中继: 接收 → 查目标 Session → 发送

二、后端设计

2.1 核心文件清单

文件	行数	位置
ChatEndpoint.java	96	<code>src/main/java/com/example/chat/websocket/ChatEndpoint.java</code>
VoiceEndpoint.java	47	<code>src/main/java/com/example/chat/websocket/VoiceEndpoint.java</code>
SocketRegistry.java	49	<code>src/main/java/com/example/chat/websocket/SocketRegistry.java</code>
HttpSessionConfigurator.java	19	<code>src/main/java/com/example/chat/websocket/HttpSessionConfigurator.java</code>

2.2 SocketRegistry — 连接注册中心

数据结构:

```
ConcurrentHashMap<String, ConcurrentHashMap<Long, Set<Session>>>
//      channel      userId      sessions
//      "chat" / "voice"
```

核心方法:

方法	功能
<code>shared()</code>	获取单例
<code>add(channel, userId, session)</code>	注册连接: <code>sessions.computeIfAbsent(userId, k -> ConcurrentHashMap.newKeySet()).add(session)</code>
<code>remove(channel, userId, session)</code>	移除连接
<code>send(channel, userId, json)</code>	向某用户的所有 session 发送消息。跳过已关闭的 session，忽略 IO 异常

线程安全:

- 外层 Map: `ConcurrentHashMap` (并发写入安全)
- 内层 Set: `ConcurrentHashMap.newKeySet()` (线程安全的 Set)
- 无额外锁

代码位置: `src/main/java/com/example/chat/websocket/SocketRegistry.java`

2.3 HttpSessionConfigurator — HTTP Session 桥接

```
@Override
public void modifyHandshake(ServerEndpointConfig config, HandshakeRequest request,
    HandshakeResponse response) {
    HttpSession httpSession = (HttpSession) request.getHttpSession();
    config.getUserProperties().put("HTTP_SESSION", httpSession);
}
```

WebSocket 握手阶段读取 HTTP Session，存入 UserProperties，使 `@OnOpen` 时可以获取用户身份。

代码位置: `src/main/java/com/example/chat/websocket/HttpSessionConfigurator.java`

2.4 ChatEndpoint — 实时消息端点

端点路径: `/ws/chat` 注册器: `@ServerEndpoint(value = "/ws/chat", configurator = HttpSessionConfigurator.class)`

消息分发逻辑 (`@OnMessage`):

```
public void message(Session socket, String raw) {
    JsonNode node = JSON.readTree(raw);
    String event = node.has("event") ? node.get("event").asText() : null;

    if ("REACTION".equals(event)) {
        handleReaction(userId, node); // 添加/移除 Reaction
    } else if ("RECALL".equals(event)) {
        handleRecall(userId, node); // 撤回消息
    } else {
        // 默认为发送普通消息
        SendMessageRequest req = JSON.treeToValue(node, SendMessageRequest.class);
        List<ChatMessage> messages = chatService.sendWithAssistantReplies(userId, req);
        // 发送给 sender + 所有 conversation recipients
        for (ChatMessage msg : messages) {
            registry.send("chat", userId, JSON.writeValueAsString(msg)); // 发送者
            for (long recipientId : chatService.recipients(conversationId, senderId)) {
```

```

        registry.send("chat", recipientId, JSON.writeValueAsString(msg)); // 接收者
    }
}
}
}

```

代码位置: `src/main/java/com/example/chat/websocket/ChatEndpoint.java` — `@OnMessage` 方法

事件类型:

event	处理函数	广播内容
REACTION	<code>handleReaction()</code>	<code>ReactionUpdate {event:"REACTION", conversationId, messageId, reactions}</code>
RECALL	<code>handleRecall()</code>	<code>RecallUpdate {event:"RECALL", conversationId, message}</code>
(default)	普通消息	<code>ChatMessage</code> 对象 (JSON)

错误处理:

- 发送失败只向发送者返回错误 JSON:
`{"event":"ERROR", "conversationId":..., "content":..., "message":...}`
- 其他接收者不受影响

代码位置: `src/main/java/com/example/chat/websocket/ChatEndpoint.java` — 错误处理段

2.5 VoiceEndpoint — 信令中继端点

端点路径: `/ws/voice` 注册器: `@ServerEndpoint(value = "/ws/voice", configurator = HttpSessionConfigurator.class)`

消息处理 (`@OnMessage`):

```

public void message(Session socket, String raw) {
    VoiceSignal signal = JSON.readValue(raw, VoiceSignal.class);
    // 将信号包装后转发给目标用户
    RoutedVoiceSignal routed = new RoutedVoiceSignal(
        userId, signal.callId(), signal.type(), signal.payload()
    );
    registry.send("voice", signal.targetUserId(), JSON.writeValueAsString(routed));
}

```

VoiceEndpoint 是纯中继:

- 接收: `{targetUserId, callId, type, payload}`
- 转发: `{fromUserId, callId, type, payload}` (添加 `fromUserId`)

信号类型 (`payload.type`):

类型	说明
call-invite	来电通知 (通过 <code>VoiceCallNotifier</code> 发送)
call-accepted	对方已接听
call-rejected	对方已拒绝

类型	说明
call-ended	通话已结束
offer	SDP offer
answer	SDP answer
(candidate)	ICE candidate

代码位置: `src/main/java/com/example/chat/websocket/VoiceEndpoint.java`

三、前端 WebSocket 集成

3.1 Chat WebSocket 客户端

```
function connectChatSocket() {
  if (chatSocket?.readyState === WebSocket.OPEN) return chatSocket;
  chatSocket = new WebSocket(chatSocketUrl()); // ws(s)://host/path/ws/chat
  chatSocket.addEventListener("message", onChatMessage);
  chatSocket.addEventListener("close", () => { /* 重连 */ });
  return chatSocket;
}
```

消息处理 (`onChatMessage`):

```
chatSocket.addEventListener("message", e => {
  const data = JSON.parse(e.data);
  if (data.event === "REACTION")      → updateReactionUI(data);
  else if (data.event === "RECALL")   → replaceMessage(data.message);
  else if (data.event === "GROUP_INVITATION") → showNotification(data);
  else if (data.event === "ERROR")    → renderFailedMessage(data);
  else                                → appendMessage(data); // 普通消息
});
```

代码位置: `src/main/webapp/assets/js/chat.js` — `connectChatSocket()` (~5-45行)

3.2 Voice WebSocket 客户端

```
function connectVoiceSocket() {
  voiceSocket = new WebSocket(voiceSocketUrl()); // ws(s)://host/path/ws/voice
  voiceSocket.addEventListener("message", onSignal);
  voiceSocket.addEventListener("close", () => { setTimeout(connectVoiceSocket, 2000); });
  return voiceSocket;
}
```

代码位置: `src/main/webapp/assets/js/voice.js` — `connectVoiceSocket()` (106-113行)

3.3 连接管理

场景	策略
正常连接	复用已有 WebSocket (单例)
连接断开	自动重连 (Chat: 无间隔, Voice: 2秒间隔)
发送失败 (WebSocket未就绪)	Chat: 降级到 REST; Voice: 静默跳过 (由polling备份)
页面关闭	Chat: 自然断开; Voice: sendBeacon 发送挂断请求

3.4 初始化时机

- Chat WebSocket: 页面加载时连接 (app.js boot 阶段)
- Voice WebSocket: 页面加载时连接 (voice.js 脚本加载时自动调用)

代码位置：

- `src/main/webapp/assets/js/chat.js` — 最后一行 `connectChatSocket()`
- `src/main/webapp/assets/js/voice.js` — 第272行 `connectVoiceSocket()`

四、数据流示意

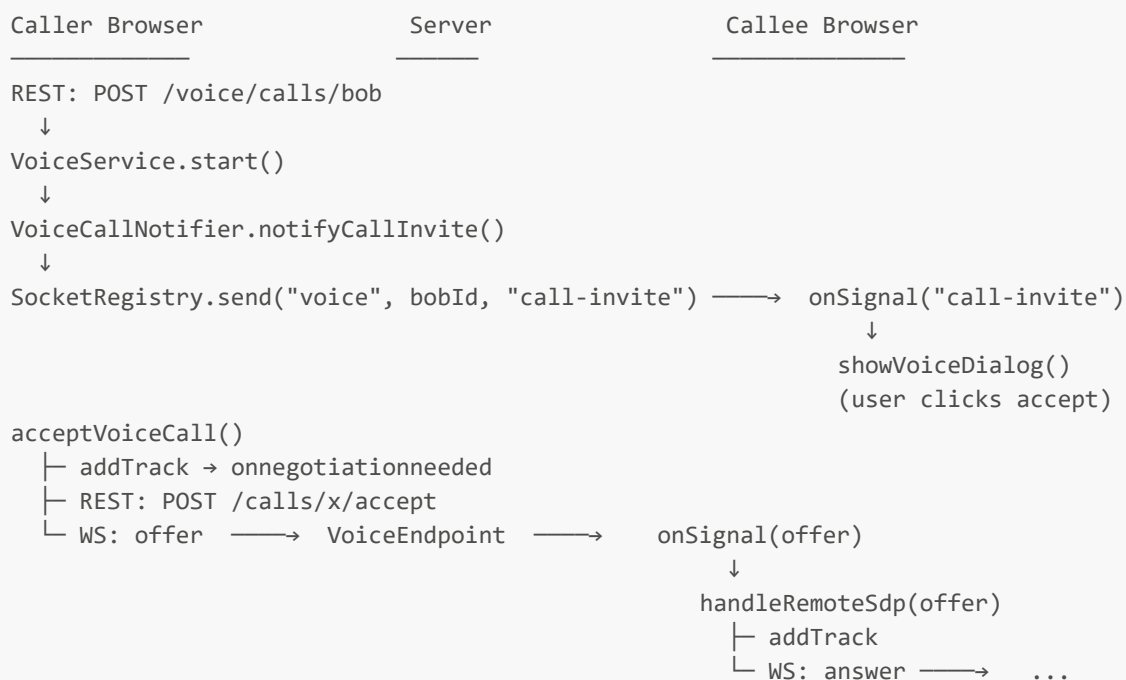
4.1 普通消息流



4.2 Reaction 流



4.3 Voice Signaling 流



双方 ICE candidates 通过 WebSocket 中继交换
双方 ontrack → 双向音频/视频建立

五、常驻连接与 socket 状态

Channel	用途	数据方向
"chat"	实时聊天	双向 (发件+收件)
"voice"	通话信令	双向 (SDP/ICE中继)

Socket 生命周期回调:

```

@OnOpen → 从 HttpSession 拿 userId → registry.add(channel, userId, session)
@OnMessage → 业务处理 → registry.send(targetChannel, targetUserId, json)
@OnClose → registry.remove(channel, userId, session)
@OnError → (由 OnClose 处理)
  
```

六、与 REST 的关系

类型	用途	降级策略
消息发送	WebSocket 优先	失败 → REST POST <code>/api/chat/messages</code>
Reaction	WebSocket 优先	失败 → REST POST <code>/api/chat/messages/{id}/reactions</code>
Recall	WebSocket 优先	失败 → REST POST <code>/api/chat/messages/{id}/recall</code>
通话信令	WebSocket + REST 双重	SDP/ICE 走 WS; 状态查询走 REST (GET <code>/voice/calls/{id}</code>) + 3秒轮询备份
会话列表	纯 REST	GET <code>/api/chat/conversations</code>
聊天历史	纯 REST	GET <code>/api/chat/conversations/{id}/history</code>

降级逻辑代码位置: `src/main/webapp/assets/js/chat.js` — `sendMessage()` (~1113行), `sendReaction()` (~809行), `recallMessage()` (~825行) 轮询备份代码位置: `src/main/webapp/assets/js/voice.js` — `setInterval` (~243-258行)